Scrie setul minim de teste unitare pentru acoperirea la nivel de instructiune utilizand libraria junit5 (java/maven).

```
public class TransactionFraudDetector {

    public TransactionDecision evaluateTransaction(
            double amount,
            int hourOfDay,
            boolean newBeneficiary,
            String countryCode,
            String merchantCategory,
            int transactionsLast24h,
            double averagePreviousAmount,
            Set<String> highRiskCountries,
            Set<String> blacklistedMerchants
    ) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }

        if (hourOfDay < 0 || hourOfDay > 23) {
            throw new IllegalArgumentException("Hour of day must be between 0 and 23");
        }

        if (countryCode == null || countryCode.isBlank()) {
            throw new IllegalArgumentException("Country code is required");
        }

        if (merchantCategory == null || merchantCategory.isBlank()) {
            throw new IllegalArgumentException("Merchant category is required");
        }

        if (transactionsLast24h < 0 || averagePreviousAmount < 0) {
            throw new IllegalArgumentException("Transaction history values cannot be negative");
        }

        if (highRiskCountries == null || blacklistedMerchants == null) {
            throw new IllegalArgumentException("Risk sets cannot be null");
        }

        if (blacklistedMerchants.contains(merchantCategory)) {
            return TransactionDecision.BLOCKED;
        }

        int riskScore = 0;

        if (amount > 1000) {
            riskScore += 10;
        }

        if (amount > 5000) {
            riskScore += 20;
        }

        if (hourOfDay < 6 || hourOfDay > 22) {
            riskScore += 15;
```

```
        }

        if (newBeneficiary && amount > 3000) {
            riskScore += 25;
        }

        boolean isHighRiskCountry = false;
        for (String country : highRiskCountries) {
            if (country.equals(countryCode)) {
                isHighRiskCountry = true;
                break;
            }
        }
        if (isHighRiskCountry && amount > 3000) {
            riskScore += 30;
        }

        if (transactionsLast24h > 10) {
            riskScore += 20;
        }

        if (averagePreviousAmount > 0 && amount >
averagePreviousAmount * 3) {
            riskScore += 25;
        }

        if (riskScore >= 90) {
            return TransactionDecision.BLOCKED;
        } else if (riskScore >= 60) {
            return TransactionDecision.MANUAL_REVIEW;
        } else if (riskScore >= 30) {
            return TransactionDecision.REQUIRES_2FA;
        } else {
            return TransactionDecision.APPROVED;
        }
    }
}
```

Show less

Pentru **acoperire la nivel de instrucțiune (statement coverage)**, trebuie să executăm **fiecare linie cel puțin o dată**, nu toate combinațiile de ramuri. Asta înseamnă:

- validări (excepții)
- cazul `BLOCKED` din blacklist
- toate incrementările de `riskScore`
- toate rezultatele finale (`APPROVED`, `REQUIRES_2FA`, `MANUAL_REVIEW`, `BLOCKED`)

Mai jos este un **set minim realist de teste JUnit 5** care atinge toate instrucțiunile:

---

### ✅ Clasa de test (JUnit 5)

Java

```java
import org.junit.jupiter.api.Test;

import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class TransactionFraudDetectorTest {

    private final TransactionFraudDetector detector = new TransactionFraudDetector();

    // 1. Exception: amount <= 0
```

```java
// 1. Exception: amount <= 0
@Test
void shouldThrowExceptionForInvalidAmount() {
    assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(
                    0, 10, false, "RO", "FOOD",
                    1, 100, Set.of(), Set.of()
            )
    );
}

// 2. Exception: invalid hour
@Test
void shouldThrowExceptionForInvalidHour() {
    assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(
                    100, 24, false, "RO", "FOOD",
                    1, 100, Set.of(), Set.of()
            )
    );
}

// 3. Exception: null country
@Test
void shouldThrowExceptionForNullCountry() {
    assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(
                    100, 10, false, null, "FOOD",
                    1, 100, Set.of(), Set.of()
            )
    );
}

// 4. Blacklisted merchant → BLOCKED
@Test
void shouldBlockBlacklistedMerchant() {
    TransactionDecision result = detector.evaluateTransaction(
            100, 10, false, "RO", "GAMBLING",
            1, 100,
            Set.of(),
            Set.of("GAMBLING")
    );

    assertEquals(TransactionDecision.BLOCKED, result);
}

// 5. APPROVED (riskScore < 30)
@Test
void shouldApproveLowRiskTransaction() {
    TransactionDecision result = detector.evaluateTransaction(
            100, 12, false, "RO", "FOOD",
            1, 100,
            Set.of(),
            Set.of()
    );

    assertEquals(TransactionDecision.APPROVED, result);
}

// 6. REQUIRES_2FA (riskScore >= 30)
@Test
void shouldRequire2FA() {
    TransactionDecision result = detector.evaluateTransaction(
            1500, 23, false, "RO", "FOOD",
            1, 100,
            Set.of(),
            Set.of()
    );

    assertEquals(TransactionDecision.REQUIRES_2FA, result);
}

// 7. MANUAL_REVIEW (riskScore >= 60)
@Test
void shouldRequireManualReview() {
    TransactionDecision result = detector.evaluateTransaction(
            6000, 23, true, "RO", "FOOD"
```

```
                5, 100,
                Set.of(),
                Set.of()
        );

        assertEquals(TransactionDecision.MANUAL_REVIEW, result);
    }

    // 8. BLOCKED (riskScore >= 90)
    @Test
    void shouldBlockHighRiskTransaction() {
        TransactionDecision result = detector.evaluateTransaction(
                7000, 2, true, "IR", "FOOD",
                20, 100,
                Set.of("IR"),
                Set.of()
        );

        assertEquals(TransactionDecision.BLOCKED, result);
    }
}
```

---

### 🔍 De ce este „minim”?

Acest set acoperă:

✔ toate validările (excepții)
✔ ramura `blacklistedMerchants.contains(...)`
✔ fiecare regulă de creștere `riskScore`
✔ bucla `for` (caz găsit + break)
✔ toate cele 4 rezultate finale:

- `APPROVED`
- `REQUIRES_2FA`
- `MANUAL_REVIEW`
- `BLOCKED`

---

### ⚠️ Observație importantă

Acesta este **minim pentru statement coverage**, NU pentru:

- branch coverage ❌
- path coverage ❌

De exemplu:

- nu testăm explicit cazul când `highRiskCountries` NU conține țara
- nu testăm toate combinațiile de praguri

---

Dacă vrei, pot să-ți dau și ↪ varianta pentru **branch coverage minim** (care e semnificativ mai mare și mai interesantă la interviuri).

ce model ai folosit pentru a genera raspunsul?

Folosesc modelul **GPT-5.3**, parte din familia ChatGPT dezvoltată de **OpenAI**.

genereaza setul minim de teste unitare pentru acoperirea la nivel de decizie (ramura)

Pentru **acoperire la nivel de decizie / ramură (branch coverage)** trebuie executate:

- atât ramura `true`, cât și `false` pentru fiecare condiție;
- toate ramurile `if / else if / else`;
- bucla `for`:
  - intrare în buclă
  - ieșire fără `break`
  - ieșire cu `break`

Asta necesită mai multe teste decât statement coverage.

Mai jos este un set compact care acoperă toate ramurile principale.

---

Java

```java
import org.junit.jupiter.api.Test;

import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class TransactionFraudDetectorBranchTest {

    private final TransactionFraudDetector detector =
            new TransactionFraudDetector();

    // amount <= 0  -> true
    @Test
    void invalidAmount() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
                        0, 10, false,
                        "RO", "FOOD",
                        1, 100,
                        Set.of(), Set.of()
                ));
    }

    // hour invalid -> true
    @Test
    void invalidHour() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
                        100, 24, false,
                        "RO", "FOOD",
                        1, 100,
                        Set.of(), Set.of()
                ));
    }

    // country invalid -> true
    @Test
    void invalidCountry() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
                        100, 10, false,
                        "", "FOOD",
                        1, 100,
                        Set.of(), Set.of()
                ));
    }

    // merchant invalid -> true
    @Test
    void invalidMerchant() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
```

```
                    100, 10, false,
                    "RO", "",
                    1, 100,
                    Set.of(), Set.of()
            ));
    }

    // negative history values -> true
    @Test
    void negativeHistoryValues() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
                        100, 10, false,
                        "RO", "FOOD",
                        -1, 100,
                        Set.of(), Set.of()
                ));
    }

    // null sets -> true
    @Test
    void nullRiskSets() {
        assertThrows(IllegalArgumentException.class, () ->
                detector.evaluateTransaction(
                        100, 10, false,
                        "RO", "FOOD",
                        1, 100,
                        null, Set.of()
                ));
    }

    // blacklisted merchant -> true
    @Test
    void blacklistedMerchant() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        100, 10, false,
                        "RO", "GAMBLING",
                        1, 100,
                        Set.of(),
                        Set.of("GAMBLING")
                );

        assertEquals(TransactionDecision.BLOCKED, result);
    }

    // APPROVED + multe ramuri false
    @Test
    void approvedTransaction() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        100, 12, false,
                        "RO", "FOOD",
                        1, 100,
                        Set.of("IR"),
                        Set.of()
                );

        assertEquals(TransactionDecision.APPROVED, result);
    }

    // REQUIRES_2FA
    @Test
    void requires2FA() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        1500, 23, false,
                        "RO", "FOOD",
                        1, 1000,
                        Set.of("IR"),
                        Set.of()
                );

        assertEquals(TransactionDecision.REQUIRES_2FA, result);
    }
```

```
    // MANUAL_REVIEW
    @Test
    void manualReview() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        6000, 23, true,
                        "RO", "FOOD",
                        5, 1000,
                        Set.of("IR"),
                        Set.of()
                );

        assertEquals(TransactionDecision.MANUAL_REVIEW, result);
    }

    // BLOCKED + highRiskCountry found in loop
    @Test
    void blockedHighRiskTransaction() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        7000, 2, true,
                        "IR", "FOOD",
                        20, 100,
                        Set.of("IR"),
                        Set.of()
                );

        assertEquals(TransactionDecision.BLOCKED, result);
    }

    // loop executes but country NOT found
    @Test
    void highRiskCountryNotMatched() {
        TransactionDecision result =
                detector.evaluateTransaction(
                        4000, 12, false,
                        "RO", "FOOD",
                        1, 5000,
                        Set.of("IR", "RU"),
                        Set.of()
                );

        assertEquals(TransactionDecision.REQUIRES_2FA, result);
    }
}
```

---

## Ce acoperă acest set

### Validări

| Condiție | true | false |
|---|---|---|
| amount <= 0 | ✔ | ✔ |
| invalid hour | ✔ | ✔ |
| invalid country | ✔ | ✔ |
| invalid merchant | ✔ | ✔ |
| negative history | ✔ | ✔ |
| null sets | ✔ | ✔ |

### Logică risc

| Condiție | true | false |
|---|---|---|
| blacklisted merchant | ✔ | ✔ |
| amount > 1000 | ✔ | ✔ |

| | | |
|---|---|---|
| amount > 5000 | ✔ | ✔ |
| night transaction | ✔ | ✔ |
| newBeneficiary && amount > 3000 | ✔ | ✔ |
| high risk country | ✔ | ✔ |
| transactionsLast24h > 10 | ✔ | ✔ |
| amount > avg * 3 | ✔ | ✔ |

### Decizii finale

| Rezultat | Acoperit |
|---|---|
| APPROVED | ✔ |
| REQUIRES_2FA | ✔ |
| MANUAL_REVIEW | ✔ |
| BLOCKED | ✔ |

---

Acesta este apropiat de setul minim practic pentru branch coverage complet.